UNM SmallSat Club

LOBO-Sat

Mission Handbook

Design, Operations, and Program Record

Version: v1.0

Date: March 1, 2026

Internal Use Document

Contents

1 Introduction **1**
1.1 Purpose . . . 1
1.2 Scope . . . 1

2 Getting Started **1**
2.1 New Member Onboarding . . . 1
2.2 How to Join a Subsystem . . . 2
2.3 Where to Ask Questions . . . 2

3 Mission Overview **3**
3.1 Mission Objectives . . . 3
3.2 Mission Requirements . . . 3
3.3 Success Criteria . . . 3
3.4 Constraints . . . 3

4 Systems Architecture **3**
4.1 Overall Architecture . . . 3
4.2 Block Diagrams . . . 3
4.3 Interfaces . . . 3
4.4 Data Flow . . . 3

5 Hardware **3**
5.1 Power System . . . 3
5.2 Radio . . . 3
5.3 ADCS . . . 3
5.4 Camera . . . 4
5.5 Telemetry . . . 4
5.5.1 Pi Set-Up . . . 4
5.5.2 IMU Hardware Set-up . . . 5
5.6 IMU Code . . . 6

6 Software and Data Systems **7**
6.1 Flight Software . . . 7
6.1.1 custom FSW . . . 7
6.1.2 cFS . . . 7
6.2 Ground Software . . . 7
6.2.1 custom . . . 7
6.3 Version Control . . . 8
6.4 Data Management . . . 8

7 Communications **8**
7.1 APRS — What We Now Know . . . 8
7.2 Legal Requirements . . . 9
7.3 Antenna Requirements . . . 9
7.4 Protocols . . . 9
7.5 Ground Station - Satellite Link . . . 9

7.6 Data Flow . . . 10
7.7 Telemetry & Data Handling . . . 10
7.8 Storage . . . 10

8 Integration & Build **11**

9 Systems Architecture **12**
9.1 Assembly . . . 12
9.2 Materials . . . 12
9.3 Thermal . . . 13
9.4 Chassis . . . 13

10 Launch, Flight, & Reentry **13**
10.1 Launch . . . 13
10.2 Flight . . . 13
10.3 Reentry . . . 13

11 Safety and Compliance **13**
11.1 Risk Management . . . 13
11.2 Safety Procedures . . . 13
11.3 Regulatory Compliance . . . 13
11.4 Export Control . . . 13

A Acronyms **13**

B References **14**

C Templates and Forms **14**

1 Introduction

1.1 Purpose

This Mission Handbook defines the design framework, operational approach, and program structure for the UNM SmallSat Club's LOBO-Sat mission. It serves as the central reference for system architecture, subsystem responsibilities, mission operations, and compliance requirements. The handbook also preserves institutional knowledge to support continuity across academic years and future missions.

1.2 Scope

This document covers the lifecycle of the LOBO-Sat, including mission objectives, system architecture, hardware and software systems, communications, integration, launch operations, recovery, and safety compliance. This document is meant to complement the Mission Requirements Document by recording implementation, operational procedures, and program status.

2 Getting Started

2.1 New Member Onboarding

Satellite Basics: The core mission of UNM SSG is to design, build, and launch satellites. Before contributing to subsystem design or implementation, every member must be able to answer: *What is a satellite?* Review this intro to satellites (5-minute read) to learn the basics of satellite operation and history.

Our Mission: The system we are developing is a balloon-launched 1U CubeSat platform (balloon satellite) intended to simulate core spacecraft architecture in a near-space environment. To understand the specific mission objectives, constraints, and success criteria for LOBO-Sat, read Sections 2 and 3 of the LOBO-Sat Mission Requirements Document (5-minute read).

Using this document: This document is written using a "coding" language called LaTeX (pronounced lah-tech), which Overleaf compiles to produce a document. LaTeX is preferred for technical and academic writing because it produces professional-quality math typesetting, automatically manages structure and citations, and cleanly separates content from formatting to keep large documents consistent and organized. Read this brief Latex intro (5-minute read) to learn the basic structure. Now practice:

- Add your name to the list below
- Uncomment the list (%)
- Compile (Ctrl + s) and look for your name in the document
- Finally, re-add the comment and recompile.

Am I Ready: Can you answer:

- What is a satellite?
- What is our mission objective?
- What is our success criteria?
- How to contribute to this document?

If you can answer all of the above, you can move on to working on a subsystem.

2.2 How to Join a Subsystem

Members must take initiative and engage with subsystem leads:

1. **Select a Track.** Decide whether you are interested in Hardware (power, radio, structure, integration) or Software (flight software, telemetry, logging, ground systems). If you prefer an administrative or managerial role, speak to Liz.
2. **Locate the Subsystem Section.** Scroll to the relevant section of this handbook (5-8) and review the subsystem descriptions.
3. **Review Current Status.** Read the subsystem section carefully to understand:
   - Current progress
   - Open technical problems
   - Immediate next steps
4. **Identify the Lead.** Each subsystem lists a Lead under the status header. Note the current lead(s) and any listed issues or next steps.
5. **Make Contact.** Message the subsystem lead on Discord or speak with them at the next meeting. Express your interest and ask how you can contribute.
6. **Define a Deliverable.** Work with the lead to define a specific, measurable task or responsibility. Every member should own a clear deliverable.
7. **If No Lead Exists.** If a subsystem or topic you are interested in does not have an active lead, consult a senior member or project lead to determine next steps. This may involve researching requirements, drafting a proposal, or initiating a new working group.

Once assigned, updates are expected at meetings to ensure progress!

2.3 Where to Ask Questions

If you have questions, don't hesitate to reach out to the group via our Discord channel!

3 Mission Overview

3.1 Mission Objectives

3.2 Mission Requirements

3.3 Success Criteria

3.4 Constraints

4 Systems Architecture

Lead(s): TBD
Status: TBD
Next Steps: TBD

4.1 Overall Architecture

4.2 Block Diagrams

4.3 Interfaces

4.4 Data Flow

5 Hardware

5.1 Power System

Lead(s): TBD
Status: TBD
Next Steps: TBD

5.2 Radio

Lead(s): TBD
Status: TBD
Next Steps: TBD

5.3 ADCS

Lead(s): TBD
Status: TBD
Next Steps: TBD

5.4 Camera

Lead(s): Liz, Jack
Status: Both Arducam and Picam are working.
Next Steps: Try an embedded systems camera similar to the Arducam; work on storing data of cam.

Arducam has great picture in comparison to the Picam; however, the nature of the Arducam being USB and having its own internal software is cumbersome as it is designed to work independently rather than in an embedded system. Opting for an Arducam more similar to the Picam in having that ribbon connection will favor the mission needs.

5.5 Telemetry

Lead(s): Liz
Status: Sparkfun IMU icm-20948 runs and reads
Next Steps: create integratable read function for cFS, integrate with LoRa

Sparkfun IMU imc-20948 is reading on all 9dof, utilizes GPIO pins 1 (3v3) , 6 (GND), 3 (SDA), and 5 (SCL). THere are I2C breakout boards that are very small by Sparkfun that can be utilized to free GPIO header room if needed. This is the link to such component : `https://www.sparkfun.com/sparkfun-qwiic-shim-for-raspberry-pi.html`

5.5.1 Pi Set-Up

The Python library for Pimoroni ICM20948 is utilized to streamline the process of using the Sparkfun ICM20948, simplifying the system embedding. The following is a detailed process of implementation. Please note that powershell was used. These commands may differ depending on the terminal.

Open the virtual environment in which your code lives (if you are using one)

```
source venv/bin/activate
```

If you need to set up a virtual environment run the following command

```
python -m venv .venv
```

Then install the github library

```
pip install icm20948
```

This should install the library. If this does not work, here is the github : `https://github.com/pimoroni/icm20948-python`

I have set up my directory to look as such...

```
icm-20948
    |
    |-- src
    |    |
    |    |- imuMain.py
```

```
|    |
|    |- readme.md
|
|-- venv
     |
     |- bin
     |
     |- include
     |
     |- lib
     |
     |- lib64
     |
     |- pyvenv.cfg
```

The imc-20948 library should be found inside

`venv\lib\`

As seen above, I created a imuMain.py file to hold the main function for the IMU. Description of this code in the IMU Code subsection below.

5.5.2 IMU Hardware Set-up

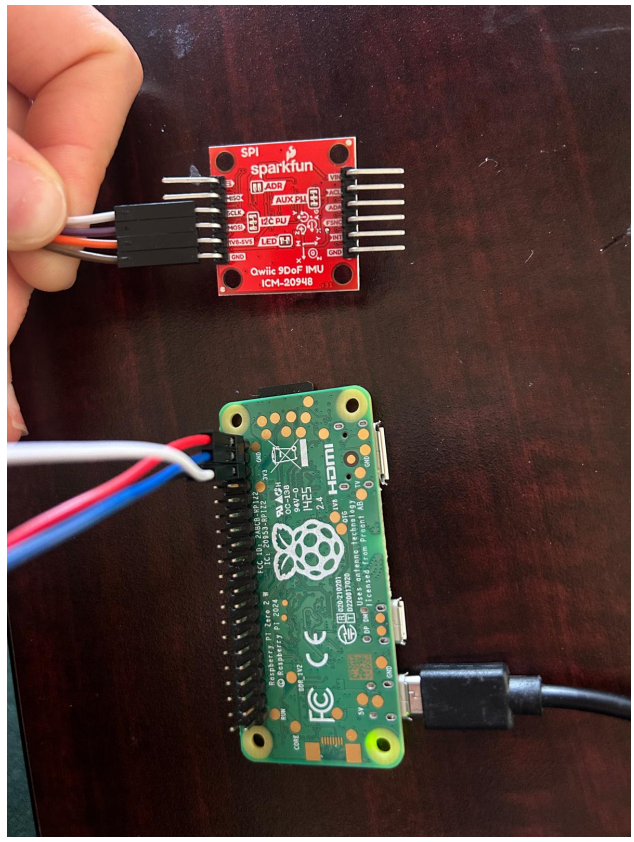
The Raspberry pi and IMU are easily connected via jumper cables, connecting the pins as such :

IMU Pins	GPIO Pins
GND	GPIO 6 (GND)
VIN	GPIO 1 (3V3)
SDA	GPIO 3 (SDA1)
SCL	GPIO 5 (SCL1)

These links were very helpful in understanding GPIO pin connections : `https://pinout.xyz/pinout/i2c` `https://forums.raspberrypi.com/viewtopic.php?t=378242`

To ease the connection process, I decided to solder the GPIO header to the pi, and also solder two 6 pin male right angle header pins to the IMU. This allows for easy plug and play with the components.

In the image, the connection can be seen, along with the soldered headers.



5.6 IMU Code

Bellow is the functioning code in imuMain.py .

```
# import time for wait and icm20948 for imu lib
import time
from icm20948 import ICM20948


# i2c at addr 69 --> used sudo i2cdetect -y 1 to find values
# if it is at 68 no change needs to be made
imu = ICM20948(i2c_addr=0x69)

# read imu data and store in params
while True:
    x, y, z = imu.read_magnetometer_data()
    ax, ay, az, gx, gy, gz = imu.read_accelerometer_gyro_data()

    # display params
    print("""
Accel: {:05.2f} {:05.2f} {:05.2f}
Gyro:  {:05.2f} {:05.2f} {:05.2f}
Mag:   {:05.2f} {:05.2f} {:05.2f}""".format(
        ax, ay, az, gx, gy, gz, x, y, z
```

```
))

# wait
time.sleep(0.25)
```

This code was created from reference of the example for the icm-20948 library to display all 9dof from the IMU `https://github.com/pimoroni/icm20948-python/blob/main/examples/read-all.py` It is recommended to view there examples and reference code files.

6 Software and Data Systems

Lead(s): Jack
Status: Developing NASA cFS Raspberry Pi
Next Steps: Connecting Ground Station to cFS, begin development of custom FSW

6.1 Flight Software

6.1.1 custom FSW

The team will develop custom flight software to manage sensor polling, telemetry packetization, data logging, and radio communication while maintaining reliable system operation throughout all mission phases.

6.1.2 cFS

The Core Flight System (cFS) is a reusable, modular flight software framework developed by NASA for embedded spacecraft applications. It provides a standardized architecture for command and data handling, message-based interprocess communication, fault management, and system monitoring[1]. It's being developed in parallel to the custom flight software that the team will use on the first launch.

6.2 Ground Software

6.2.1 custom

The team will develop and operate a custom ground station to receive, decode, log, and visualize telemetry in real time while issuing command uplinks during flight operations.

6.3 Version Control

6.4 Data Management

7 Communications

Lead(s): Julien, David, Sofia, Sam
Status: LoRa Link Verified, APRS Architecture Researched, SX1262 Eval Pending, Integration Not Started
Two Feather M0 modules have been used to send small data packets over the 915MHz band, using Arduino IDE [requires configuration of Arduino IDE to accommodate the module, and downloading the RadioHead Library, instructions available on Adafruit Resource site: https://learn.adafruit.com/adafruit-feather-m0-radio-with-lora-radio-module]
LoRa link confirmed working between two Feather M0 boards. RSSI around -54 to -55 dBm indoors. COM port re-enumerated correctly when swapping roles. Battery-only operation confirmed (3.7V 1200mAh LiPo). RSSI is signal strength only — packet loss measured via missing sequence numbers. Baofeng radios cannot receive LoRa (different modulation).
The SX1262 Eval Boards will be tested on Monday 3/2 by David and Julien. Unlike the Feather M0, the eval boards have the SX1262, which is needed for the actual mission. We found some hidden datasheets, and think we can actually use it, unlike previously believed. Ideally, if we are able to get them to work, these boards are perfectly constructed for the Antenna team to design and test for. We hope to copy the delicate PCB design of the eval boards and add an MCU and IMU to it for final launch.
Next Steps: SX1262 testing, APRS hardware decision, range testing

1. SX1262 Eval Boards will be connected to Arduino MEGAs using RadioHead library on 3/2/26
2. Selection for final flight computer (Pi vs Arduino Mega vs Feather) needed
3. APRS architecture researched but hardware not yet selected (see below)
4. Define telemetry packet format (byte structure, update rate, CRC, command structure)
5. Measure RSSI, SNR, packet loss at increasing distance
6. If APRS selected: team members need Amateur Radio licenses (Technician minimum)

7.1 APRS — What We Now Know

None of this has been built yet — this is the knowledge base for when we make the hardware decision.

APRS is NOT something a LoRa chip can do. APRS requires three layers:
1. Packet generation — AX.25 formatting with callsign, position, telemetry
2. AFSK modulation — encoding packets as 1200/2200 Hz audio tones (Bell 202)
3. 144.390 MHz FM transmission — a VHF transmitter that puts those tones on-air

Layer 1 (Packet generation): Arduino Mega ATmega2560 (MCU) or T-TWR-ESP32 (MCU)
Layer 2 (AFSK modulation): Arduino Mega ATmega2560 (MCU) or T-TWR-ESP32 (MCU)
Layer 3 (VHF FM transmission): HX1 (APRS VHF FM transmitter) or T-TWR-SA868 (VHF FM transceiver)

MCU handles Layers 1-2 in software. Radio chip handles Layer 3.

IMPORTANT: SX1262 has nothing to do with APRS. Even if you send APRS-formatted data over LoRa, it is not on 144.390 MHz FM. The SX1262 works only as its own LoRa module on 915 MHz.

Radiometrix HX1 — 300mW FM transmitter on 144.390 MHz. RF stage only (Layer 3). Needs external AFSK input from MCU and 5V supply. No GPS or MCU.

LilyGO T-TWR Plus VHF [T-TWR-ESP32 + T-TWR-SA868] (134-174 MHz) — tunes to 144.390 MHz. ESP32 handles Layers 1-2, SA868 handles Layer 3. All-in-one APRS tracker with firmware. No HX1 needed. IMPORTANT: also comes in UHF version (400-480 MHz) which CANNOT do U.S. APRS. Must get VHF version.

Architecture A — Mega + HX1:
ATmega2560 builds packet (L1) → encodes AFSK (L2) → audio wired to HX1 → HX1 transmits 144.390 MHz (L3). Needs 5V reg for HX1.

Architecture B — T-TWR Plus VHF:
T-TWR-ESP32 builds packet (L1) → encodes AFSK (L2) → T-TWR-SA868 transmits 144.390 MHz (L3). One board, flash firmware, attach GPS + antenna.

Architecture C — LoRa only (no APRS):
SX1262 → LoRa 915 MHz → private ground station. Legal unlicensed. Not on APRS network.

Decision: private telemetry only → LoRa sufficient. Public APRS tracking → need 144.390 FM transmitter. Dual redundancy → both systems, separate RF chains, separate antennas.

7.2 Legal Requirements

LoRa 915 MHz: legal under FCC Part 15, unlicensed ISM band. No callsign required.
APRS 144.390 MHz: requires Amateur Radio license (Technician minimum). Must transmit callsign. Part 97 rules.
There is no unlicensed APRS in the United States.

7.3 Antenna Requirements

Each radio system needs its own antenna. Do not share between bands.
915 MHz quarter-wave: approximately 78 mm
144.390 MHz quarter-wave: approximately 49-52 cm

7.4 Protocols

Feather M0 will not be used on the actual mission — its transceiver (SX1276/RFM95) is worse rated than the SX1262 and would not have the sensitivity needed for final link.

LoRa (primary flight link): chirp spread spectrum modulation on 915 MHz ISM
CONTROL (flight): SX1262-based transceiver (eval board now, custom PCB later). Feather M0 w/ RFM95 for dev/prototyping only.
CONTROL (ground): Feather M0 LoRa or RFM95 breakout + host (receive/parse, send commands)

7.5 Ground Station - Satellite Link

Two independent receive chains required — one for LoRa, one for APRS. Separate antennas, separate receivers, both feed into same laptop.

LoRa Ground Station (915 MHz):
ANTENNA: 915 MHz Yagi (directional gain for longer range)
RECEIVER: Feather M0 LoRa or RFM95 breakout + host as LoRa modem
RF MONITORING: RTL-SDR Blog V4 (spectrum monitoring; not a LoRa modem)
DECODING: Laptop receives telemetry from LoRa modem over USB
USB OTG adapters/cables

APRS Ground Station (144.390 MHz):
ANTENNA: 144 MHz Yagi (separate physical antenna — different band, cannot share with LoRa Yagi)
RECEIVER: Baofeng handheld tuned to 144.390 MHz, connected to Yagi
DECODING: Baofeng cannot decode APRS — plays raw AFSK tones only. Audio cable from Baofeng headphone jack → laptop running Dire Wolf (free, open-source software TNC). Dire Wolf decodes AFSK back into AX.25 packets with callsign, GPS position, telemetry. Can forward to YAAC or Xastir for live map tracking.

Summary:
915 MHz Yagi → LoRa modem → USB → Laptop
144 MHz Yagi → Baofeng → audio → Laptop/Dire Wolf

7.6 Data Flow

SENSORS: GPS modules (NEO-M8N / NEO-M9N), MPU-9255 IMU
FLIGHT CONTROL/COMPUTE: Raspberry Pi Zero 2 or Arduino Mega or Feather M0
RADIOS:
(flight LoRa): SX1262 eval board (target) or Feather M0 LoRa (dev fallback), driven by Pi/Mega
(flight APRS): T-TWR VHF (self-contained) or HX1 driven by flight computer
(ground LoRa): Feather M0 LoRa or RFM95 setup
(ground APRS): Baofeng + Yagi + laptop running Dire Wolf

7.7 Telemetry & Data Handling

- Raspberry Pi Zero 2 (if flight computer)
Logging: write to microSD (CSV or binary)
Packet parsing + timestamping
Interfaces: UART/I2C/SPI + easier file system
  - Elegoo Mega R3 – ATmega2560 (if flight computer)
Sensor polling + telemetry formatting
Radio control via SPI
Storage is NOT built-in (needs SD module or external logger)
  - Feather M0 (if flight computer)
Great for radio + simple telemetry
Storage depends on add-ons (not inherently "big logging")

7.8 Storage

- Option 1 ("Definite onboard storage"):
Pi Zero 2 is the flight computer
Logs everything to microSD
Radio sends a smaller "telemetry subset" live

- Option 2 (Mega as flight computer):
Mega + SD card module (SPI)
Write compact records (binary or CSV with careful buffering)
Radio is controlled by Mega (SPI)
- Option 3 (split-brain):
Mega reads sensors + makes packets
Pi logs + handles "heavy" storage

8 Integration & Build

9 Systems Architecture

Lead(s): Kameron, Brant
Status: Waiting for hardware dimensions
Next Steps: Input hardware models into the CAD assembly, run thermal simulations, and simulate reentry forces on completed model.

9.1 Assembly

- The main assembly are documented as "1U Assembly V0.00"
- The sub-assemblies are documented as "1U Assembly description sub X.V01.01."
  - Sub specifies that it is a sub-assembly
  - X is the Category of the sub-assembly.
  - V01 denotes the version of the assembly.
  - .01 denotes the iteration of the version.

9.2 Materials

- The materials used for the chassis:
  - 6061 Aluminum
  - ABS Plastic
  - XPS Foam
- The parts made of 6061 Aluminum:
  - 1U_Panel_C.M.01.03
  - 1U_Lid_C.M.01.02
  - 1U_BottomLid_C.M.01.01
- The parts made of ABS plastic:
  - 1U_Rail_C.M.01.05
  - 1U_TechMainShelf_E.M.01.04
  - 1U_TechRail_E.M.01.04
  - 1U_TechBottomShelf_E.M.01.01
  - 1U_TechSideShelf_E.M.01.04
- The parts made of XPS foam:
  - 1U_XPS_Foam_C.M.01.03
  - 1U_XPS_FoamBottom_C.M.01.03

9.3 Thermal

- Thermal simulations are run through SolidWorks using the "Thermal Simulation Wizard" tool.
- The assemblies are tested at expected low Stratospheric conditions. These conditions are:
  - Initial Temperature: 290K
  - Bulk Ambient Atmospheric Temperature: 220K
  - Convection Coefficient: $7.5\,\mathrm{W}/(\mathrm{m}^2 \cdot \mathrm{K})$
  - Emissivity: 0.1
  - Heat Power: 0.6W

9.4 Chassis

10 Launch, Flight, & Reentry

Lead(s): TBD
Status: TBD
Next Steps: TBD

10.1 Launch

10.2 Flight

10.3 Reentry

11 Safety and Compliance

Lead(s): TBD
Status: TBD
Next Steps: TBD

11.1 Risk Management

11.2 Safety Procedures

11.3 Regulatory Compliance

11.4 Export Control

A Acronyms

ADCS	Attitude Determination and Control System
EPS	Electrical Power System

B References

References

[1] NASA, "Core Flight System (cFS)." https://github.com/nasa/cFS, 2026. Accessed: 2026-02-13.

C Templates and Forms

Include internal forms.